

Lab 5 Visualisation Heatmap

2026-01-16

Lab 5 Heatmaps and Positioning

Goal

Create a calendar-style heatmap by aggregating total distance by Year and Week.

Imports

```
library(dplyr)
library(plotly)
library(ggplot2)

file_id <- "1N1UERM9IGhJ29apnDalVLhx9EJBmvH-v"
url <- paste0("https://drive.google.com/uc?id=", file_id)

distancevpac <- read.csv(url)

file_id <- "1lgjURcPZaIDCqBa_bjbkmZk9i12M9x9W"
url <- paste0("https://drive.google.com/uc?id=", file_id)

trailing365 <- read.csv(url)

file_id <- "1MLylJ6cizJlTFVi_FdiF11pjpT3brls0"
url <- paste0("https://drive.google.com/uc?id=", file_id)

monthly_stats <- read.csv(url)

file_id <- "1K-PNkRuAPaVWP_fh8iOZfJyulPvcK0ff"
url <- paste0("https://drive.google.com/uc?id=", file_id)

daily_summary <- read.csv(url)

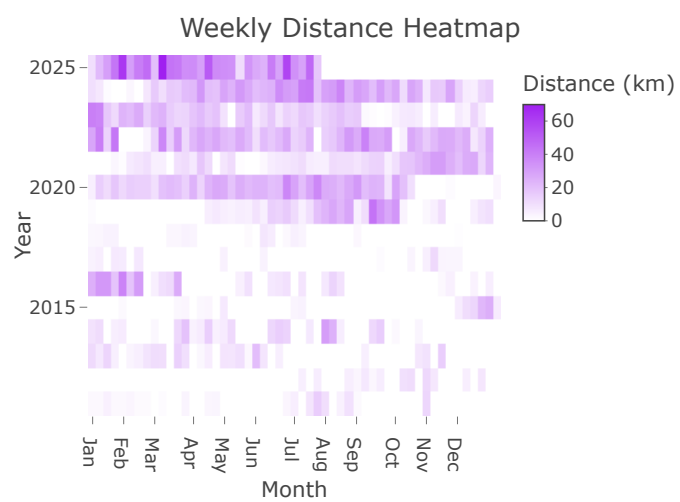
file_id <- "11J2STNOUfiyyo1FRn-zZ0QXCW85m3YFN"
url <- paste0("https://drive.google.com/uc?id=", file_id)

weekly_sum <- read.csv(url, check.names = FALSE) # Check names prevents r from adding an 'X'
#Column names should not really be numbers.

ncol(matrix_df)
tail(colnames(matrix_df), 10)
```

1. Create heatmap.

In plotly heatmaps are as straight forward as setting `x,y,z` along with `'type="heatmap"`. Take some time to format the `hovertemplate` and axis labels.



2. Create dashboard

I've included previous weeks' code in the appendix. Using that (or your own plots), add the heatmap to the bottom of the dashboard.

One convenient method is to nest subplots by building each row separately, then combining rows into a single figure:

```
row1 <- subplot(fig1, fig2, nrow = 1)
row2 <- subplot(fig3, fig4, nrow = 1)
dashboard <- subplot(row1, row2, nrow = 2)
```

This approach lets you control how many charts appear in each row (for example, a heatmap can span the full width on the final row). Note that you may still need to set arguments such as `nrow`, `titleX`, and `titleY` when combining plots.

Once you have all charts in a single figure, you can start formatting. This step can be time-consuming (layout tweaks are a normal part of building dashboards). Also note that the RStudio Viewer pane is quite small, so it is often easier to open the dashboard in a web browser. You can conveniently do this from RStudio using 'show in new window'

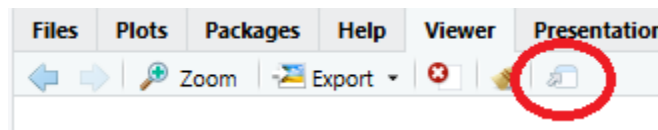


Figure 1: Fulllayout

If you experience rendering issues, try opening the output in a Chromium-based browser (Chrome or Edge), as these tend to be the best supported.



Figure 2: Fulllayout

Aim to get all charts visible; we will add annotations and subplot titles in the next lab. Remember that domain-based charts (like pie charts) may need manual placement:

```
domain = list(
  x = c(0.00, 0.48),
  y = c(0.8, 0.95)
),
```

The x and y ranges are proportions of the available space (0 to 1), where y increases from bottom to top. In some cases you may also want to manually position the heatmap colour bar, for example:

```
colorbar = list(x = 1.02, y = 0.40)
```

Appendix

```
#Pie Chart
pie_fig <- plot_ly(
  data = daily_summary,
  labels = ~AMPM,
  values = ~count,
  type = "pie",
  #Notice domain this specifies the position of the plot.
  domain = list(
    x = c(0.00, 0.48),
    y = c(0.52, 1.00)
  ),
  textinfo = "percent+label",
  textposition = "inside",
  hovertemplate = "%{label}: %{value}<extra></extra>",
  marker = list(colors = c("AM" = "#E6E6FA", "PM" = "rebeccapurple", "Both" = "mediumpurple"))[daily_summ
)

#Trailing 365 chart
trailing365$Date <- as.Date(trailing365$Date)
trailing365$hover_text <- paste0(
  "Date: ", trailing365$Date,
  "<br>Trailing365: ", sprintf("%.2f", trailing365$Trailing365)
)

p_trailing <- ggplot(trailing365, aes(x = Date, y = Trailing365, group = 1, text = hover_text)) +
  geom_line(colour = "purple") +
  labs(title = "Trailing 365-Day Kilometers",
    x = "Days", y = "Kilometers in past 365 days")

trail_fig <- ggplotly(p_trailing, tooltip = "text")

#Pace and Distance Scatter
p_scatter <- ggplot(distancevpace, aes(x = Distance_km, y = Pace_min_km)) +
  geom_point(colour = "purple") +
  labs(
    x = "Distance (km)",
```

```

    y = "Pace (min/km)"
  )

scatter_fig <- ggplotly(p_scatter)

#Bar Chart
bar_fig <- plot_ly(
  data = monthly_stats,
  x = ~YearMonth,
  y = ~(TotalDistance / 1000),
  type = "bar",
  marker = list(color = "rebeccapurple"),
  hovertemplate = paste(
    "Month: %{x}",
    "<br>Distance: %{y:.2f} km",
    "<extra></extra>"
  )
)

```

```

dash <- subplot(
  pie_fig, trail_fig,
  scatter_fig, bar_fig,
  nrows = 2,
  margin = 0.06,
  titleX = TRUE,
  titleY = TRUE
) %>%
  layout(title = list(text = "Running Dashboard"),showlegend = FALSE)

dash

```

Running Dashboard

